



Lab 2: Feature detection

MCHA4400

Semester 2 2024

Introduction

In this lab, you will become familiar with navigating the OpenCV API documentation (<https://docs.opencv.org/4.10.0/>), and use it to employ the following methods for finding features in images:

1. Harris corner detector [1],
2. Shi & Tomasi corner detector [2],
3. FAST feature detector [3],
4. ArUco marker detector [4].

You will create a C++ terminal application with a command-line interface that can open a video or image file to find and display image features. To achieve this, you will use OpenCV to do the following:

- Load images data from image and video files with appropriate input checks,
- Modify images with marker and text overlays,
- Display images and associated features, and export them to files.

The lab should be completed within 4 hours. The assessment will be done within your enrolled lab session(s). Once you complete the tasks, call the lab demonstrator to start your assessment.

The lab is worth 5% of your course grade and is graded from 0–5 marks.



GenAI Tip

You are strongly encouraged to explore the use of Large Language Models (LLMs), such as GPT, Gemini or Claude, to assist in completing this activity. If you do not already have access to an LLM, bots are available to use via the UoN Mechatronics Slack team, which you can find a link to from Canvas. If you are unsure how to make the best use of these tools, or are not getting good results, please ask your lab demonstrator for advice.

Command line interface

For the tasks specified in Sections 1–4, a command line interface is used. For this task, the OpenCV command line parser `cv::CommandLineParser` is used. The command line interface enables the user to do the following:

- a) Access useful *help* documentation for the application input arguments.
- b) Specify the maximum number of features to search for using the `-N` argument.
- c) Specify the path to the image or video that the application will process, that can be read using `cv::imread` or `cv::VideoCapture`, respectively.
- d) Specify to display the processed images on screen using `cv::imshow` or export the processed image or video using the `-e` or `--export` flags using `cv::imwrite` or `cv::VideoWriter`, respectively. Each output file must be written to the output directory with path format `out/<input-file-stem>_<detector>.<input-file-extension>`.

- e) Specify the detection algorithm to be used (`harris`, `shi`, `fast`, `aruco`) using the `-d` or `--detector` argument.
- f) Benchmark the performance of all detectors using the `-b` or `--benchmark` flags.

The interface should print relevant information to the console such as image dimensions, the number of features that were detected, and any other information specified in the following tasks.

To begin, prepare the build directory for this lab and change to it as follows:

Terminal

```
nerd@basement:~/MCHA4400/lab2$ cmake -G Ninja -B build && cd build
[cmake-ing intensifies]
nerd@basement:~/MCHA4400/lab2/build$
```

1 Harris corner detector (1 mark)

Use the OpenCV implementation of the Harris [1] corner detector, `cv::cornerHarris`, to find corners of interest in the image. Store all pixels and their associated Harris scores, which are above a chosen threshold, inside a `std::vector`¹. Adjust algorithm parameters to ensure a usable result.

- a) Plot all the features that are above some texture based threshold using `cv::circle`.
- b) Print a sorted list of the texture values and indices of the `N` most highly textured features.
- c) Plot the indices of the `N` most highly textured features using `cv::putText`.

Build and run the `lab2` program with the Harris corner detector to generate a figure similar to Figure 1 as follows:

Terminal

```
nerd@basement:~/MCHA4400/lab2/build$ ninja && ./lab2 ../data/box.jpg -d=harris -N=4
[Ninja-ing intensifies]
Using harris feature detector
Image width: 1280
Image height: 1024
Features requested: 4
Features detected: 143
  idx: 0    at point: (356,422)    Harris Score: 0.00187296 NOTE: Your scores may be different
  idx: 1    at point: (356,423)    Harris Score: 0.0017795
  idx: 2    at point: (520,203)    Harris Score: 0.00147312
  idx: 3    at point: (910,362)    Harris Score: 0.00147124
```

Experiment with the detector using the files included in the `data` directory and then run the following to export some of the results to the `out` directory:

Terminal

```
nerd@basement:~/MCHA4400/lab2/build$ ./lab2 -d=harris -N=15 -e ../data/lab.jpg
nerd@basement:~/MCHA4400/lab2/build$ ./lab2 -d=harris -N=15 -e ../data/descriptor0.JPG
nerd@basement:~/MCHA4400/lab2/build$ ./lab2 -d=harris -N=15 -e ../data/outdoor.MOV
```

¹Consider using a `std::vector` of structures. The structure could comprise of elements `x`, `y`, `score` which contain the pixel coordinates `x`, `y` and the score respectively.

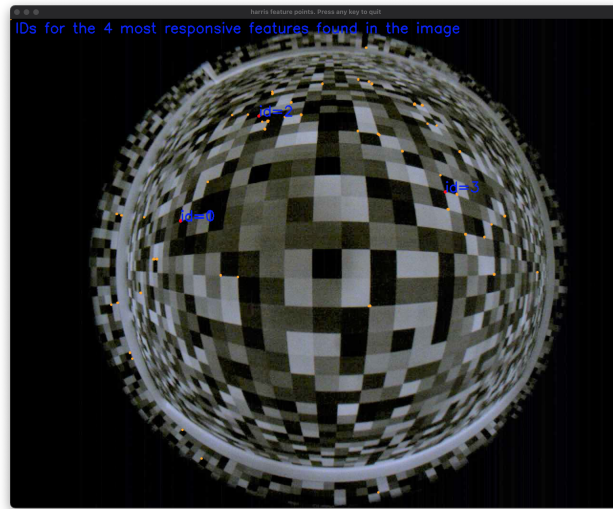


Figure 1: Output from running `./lab2 ../data/box.jpg -d=harris -N=4`

2 Shi & Tomasi corner detector (1 mark)

Implement the Shi & Tomasi [2] corner detector using `cv::cornerMinEigenVal` to find corners of interest in the image. Do **not** use `cv::goodFeaturesToTrack`, as the keypoint scores cannot be extracted to sort features according to their texture. Store all pixels and their associated eigenvalues, which are above a chosen threshold, inside a `std::vector`. Adjust thresholds and other algorithm parameters to ensure a usable result.

- Plot all the features that are above some texture based threshold using `cv::circle`.
- Print a sorted list of the texture values and indices of the `N` most highly textured features.
- Plot the indices of the `N` most highly textured features using `cv::putText`.

Experiment with the detector using the files included in the `data` directory and then run the following to export some of the results to the `out` directory:

Terminal

```
nerd@basement:~/MCHA4400/lab2/build$ ./lab2 -d=shi -N=15 -e ../data/lab.jpg
nerd@basement:~/MCHA4400/lab2/build$ ./lab2 -d=shi -N=15 -e ../data/descriptor0.JPG
nerd@basement:~/MCHA4400/lab2/build$ ./lab2 -d=shi -N=15 -e ../data/outdoor.MOV
```

3 FAST feature detector (1 mark)

Implement the FAST (Features from Accelerated Segment Test) feature detector using `cv::FAST` or `cv::FastFeatureDetector` to find corner features in the image. Store all detected keypoints and their associated scores inside a `std::vector`. Adjust algorithm parameters to ensure a usable result.

- Plot all the detected features using `cv::circle`.
- Print a sorted list of the feature scores and indices of the `N` strongest features.

- c) Plot the indices of the N strongest features using `cv::putText`.

Build and run the `lab2` program with the FAST feature detector as follows:

Terminal

```
nerd@basement:~/MCHA4400/lab2/build$ ninja && ./lab2 ../data/box.jpg -d=fast -N=4
[Ninja-ing intensifies]
Using fast feature detector
Image width: 1280
Image height: 1024
Features requested: 4
Features detected: 160
  idx: 0    at point: (188,562)    FAST Score: 160
  idx: 1    at point: (189,564)    FAST Score: 155
  idx: 2    at point: (629,715)    FAST Score: 150
  idx: 3    at point: (817,895)    FAST Score: 143
```

Experiment with the detector using the files included in the `data` directory and then run the following to export some of the results to the `out` directory:

Terminal

```
nerd@basement:~/MCHA4400/lab2/build$ ./lab2 -d=fast -N=15 -e ../data/lab.jpg
nerd@basement:~/MCHA4400/lab2/build$ ./lab2 -d=fast -N=15 -e ../data/descriptor0.JPG
nerd@basement:~/MCHA4400/lab2/build$ ./lab2 -d=fast -N=15 -e ../data/outdoor.MOV
```

4 ArUco marker detector (1 mark)

Use the OpenCV implementation of the ArUco [4] library to find markers with `cv::aruco::ArucoDetector::detectMarkers` in the image using the `cv::aruco::DICT_6X6_250` dictionary.

- Print a sorted list of the marker corner locations, sorted by the marker identifiers.
- Plot ArUco markers using `cv::aruco::drawDetectedMarkers`.



GenAI Tip

The API for the ArUco module changed significantly with the release of OpenCV 4.7 on 29th Dec 2022. These changes may be more recent than the training cut-off for some of the available LLMs. The [ArUco OpenCV tutorial](#) may be a good reference instead.

Build and run the `lab2` program with the ArUco feature detector as follows:

Terminal

```
nerd@basement:~/MCHA4400/lab2/build$ ninja && ./lab2 ../data/singlemarkersoriginal.jpg -d=aruco
[Ninja-ing intensifies]
Using aruco feature detector
Image width: 640
Image height: 480
ID: 23 with corners: (298,185) (334,186) (335,212) (297,211)
ID: 40 with corners: (359,310) (404,310) (409,351) (362,350)
ID: 62 with corners: (233,273) (190,273) (196,241) (237,241)
ID: 98 with corners: (427,255) (469,256) (477,289) (434,288)
ID: 124 with corners: (425,163) (430,186) (394,186) (390,162)
ID: 203 with corners: (195,155) (230,155) (227,178) (190,178)
```

Experiment with the detector using the files included in the `data` directory and then run the following to export some of the results to the `out` directory:

Terminal

```
nerd@basement:~/MCHA4400/lab2/build$ ./lab2 -d=aruco -e ../data/negativeAdrian.png
nerd@basement:~/MCHA4400/lab2/build$ ./lab2 -d=aruco -e ../data/singlemarkersoriginal.jpg
nerd@basement:~/MCHA4400/lab2/build$ ./lab2 -d=aruco -e ../data/tags.png
```

5 Benchmark the results (1 mark)

Complete the implementation of the benchmarking code in `src/main.cpp` to compare the performance of all 4 feature detectors using `ankerl:nanobench`.

Configure `nanobench` to produce an output similar to the following, showing the benchmark results relative to the Harris detector:

Terminal

```
nerd@basement:~/MCHA4400/lab2/build$ ninja && ./lab2 --benchmark -N=15 ../data/lab.jpg
[Ninja-ing intensifies]
Benchmark results:
| relative |          ns/op |          op/s |    err% |    total | Feature detector
|-----:|-----:|-----:|-----:|-----:|-----:
| 100.0% |    x,xxx,xxx.xx |    xxx.xx |    x.x% |    x.xx | 'Harris'
| xxx.x% |    x,xxx,xxx.xx |    xxx.xx |    x.x% |    x.xx | 'Shi-Tomasi'
| xxx.x% |    x,xxx,xxx.xx |    xxx.xx |    x.x% |    x.xx | 'FAST'
| xxx.x% |    x,xxx,xxx.xx |    xxx.xx |    x.x% |    x.xx | 'ArUco'
```

Based the quality of features detected and their runtime performance, which of the three point feature detectors would you recommend?

References

- [1] Christopher Harris and Mike Stephens. A combined corner and edge detector. In *Alvey vision conference*, volume 15, pages 10–5244. Citeseer, 1988.
- [2] Jianbo Shi and Carlo Tomasi. Good features to track. In *1994 Proceedings of IEEE Conference on Computer Vision and Pattern Recognition*, pages 593–600, 1994.

- [3] Edward Rosten and Tom Drummond. Machine learning for high-speed corner detection. In *Computer Vision—ECCV 2006: 9th European Conference on Computer Vision, Graz, Austria, May 7–13, 2006. Proceedings, Part I 9*, pages 430–443. Springer, 2006.
- [4] S. Garrido-Jurado, R. Muñoz-Salinas, F.J. Madrid-Cuevas, and M.J. Marín-Jiménez. Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, 47(6):2280–2292, 2014. ISSN 0031-3203. doi: <https://doi.org/10.1016/j.patcog.2014.01.005>. URL <https://www.sciencedirect.com/science/article/pii/S0031320314000235>.